

REFERAT PROIECT SGBD

CINETRACK — PLATFORMA DE GESTIUNE SI ANALIZA A FILMELOR

Student: **Paraschiv Victor Andrei**

Anul II, Facultatea de Informatica, UAIC Iasi

1. De ce am ales sa construiesc asta asa

Cand am citit prima data cerinta acestui proiect, primul gand a fost sa ma apuc direct de cod. Am deschis IntelliJ, am creat un proiect Spring Boot si eram gata sa las Hibernate-ul sa imi genereze tabelele in baza de date. Apoi m-am oprit. Mi-am dat seama ca daca nu controlez eu structura datelor, legaturile si constrangerile direct de la sursa, toata aplicatia o sa fie un castel de carti de joc. Asa ca am inchis IDE-ul si am luat o coala de hartie ca sa desenez arhitectura.

Proiectul CineTrack este o platforma web de gestiune a vizionarilor de filme, dar nu am vrut sa fie un simplu tabel in care adaugi inregistrari. Am vrut un sistem care “simte” ce face utilizatorul: administreaza catalogul de filme si versiunile video, gestioneaza actorii cu recenzii specifice pe rol, si urmareste istoricul clientilor (vizionari, voturi, etichete emotionale bifate).

Pe partea tehnica, am luat o decizie dura dar asumata: baza de date Oracle face toata munca grea analitica prin proceduri PL/SQL, backend-ul de Spring Boot este scris cu JDBC absolut pur (fara ORM-uri care ascund query-urile), iar frontend-ul din Angular 18 foloseste CSS scris manual pentru un “Dark Mode” autentic, ignorand framework-uri greoaie ca Bootstrap.

2. Cerintele de business vazute prin ochii clientului

Inainte sa creez vreo tabela, am scris regulile din perspectiva platformei. Un film nu e doar un titlu, e o entitate care strange vizionari si, mai ales, un rating. Acest rating nu e o valoare pusa de un admin, ci o medie calculata organic in fundal la fiecare actiune a clientilor. Apoi, am tinut cont de realitate: nu ne uitam “la un film”, ne uitam la un film in format 4K subtitrat, sau HD dublat. Deci a trebuit sa modelez versiunile filmelor ca entitati care se pot viziona si alege individual.

Despre actori, am vrut ca platforma sa reflecte realitatea critica: nu dai nota unui actor ca om, ci dai nota prestatiei lui dintr-un anumit film. Un actor poate face un rol genial in *Inception*, dar unul mai putin bun in alt film. De aceea, am gandit posibilitatea de a atasa recenzii la asocierea exacta dintre un actor si un film.

In ceea ce priveste experienta clientului, am modelat un istoric foarte granular: cand s-a uitat, la ce versiune s-a uitat, daca a terminat filmul sau l-a oprit la jumatate. Cand voteaza, pe langa o nota clasica, clientul poate bifa si “tag-uri” predefinite (ex: “emotionant”, “scenariu slab”, “as recomanda”). Combinatia dintre nota, aceste etichete si cuvintele din comentariul text reprezinta sursa de date pentru un algoritm integrat care calculeaza automat daca acel client are un sentiment pozitiv sau negativ, permitand platformei sa ii genereze recomandari inteligente.

3. De la o relatie universala la forma normala BCNF

Asezarea bazei de date a fost de departe cel mai riguros proces. Am pornit de la ideea unei singure relatii universale mamut, R, care continea tot: id-ul clientului, datele personale, datele filmului, ratingul, detaliile versiunii, detaliile actorului, nota acordata, textul comentariului, etc. Evident, aveam o explozie de redundanta si anomalii.

Am aplicat regulile normalizarii pe baza dependentelor functionale. De exemplu, id-ul filmului determina titlul si categoria. Id-ul versiunii determina formatul si limba. O dependenta puternic compusa este cea a unui vot: doar combinatia (client_id, film_id) determina scorul si comentariul. Aplicand testul BCNF, am observat ca in relatia universala R, determinanti ca film_id nu erau superchei.

Descompunerea s-a facut natural: am extras entitatile puternice (FILMS, CLIENTS, ACTORS) unde ID-ul generat a devenit cheie primara si supercheie. Apoi am separat FILM_VERSIONS ca entitate slaba (un format 4K in germana depinde direct de filmul respectiv). Pentru asocierile cu dependente compuse am creat tabelele M:N: FILM_ACTORS (unde intra detaliile rolului), VOTES (pentru recenzii, cu restrictie unica pe perechea client-film) si VIEWINGS (pentru istoric). Nomenclatorul fix VOTE_TAGS l-am pus separat si l-am legat de voturi prin CLIENT_TAG_SELECTIONS.

In final, am ajuns la 9 tabele curate in care, pentru absolut orice dependenta functionala, partea stanga este o supercheie a acelei relatii. Baza de date este complet in Forma Normala Boyce-Codd.

4. Schema E/A, cheile sintetice si protectia datelor

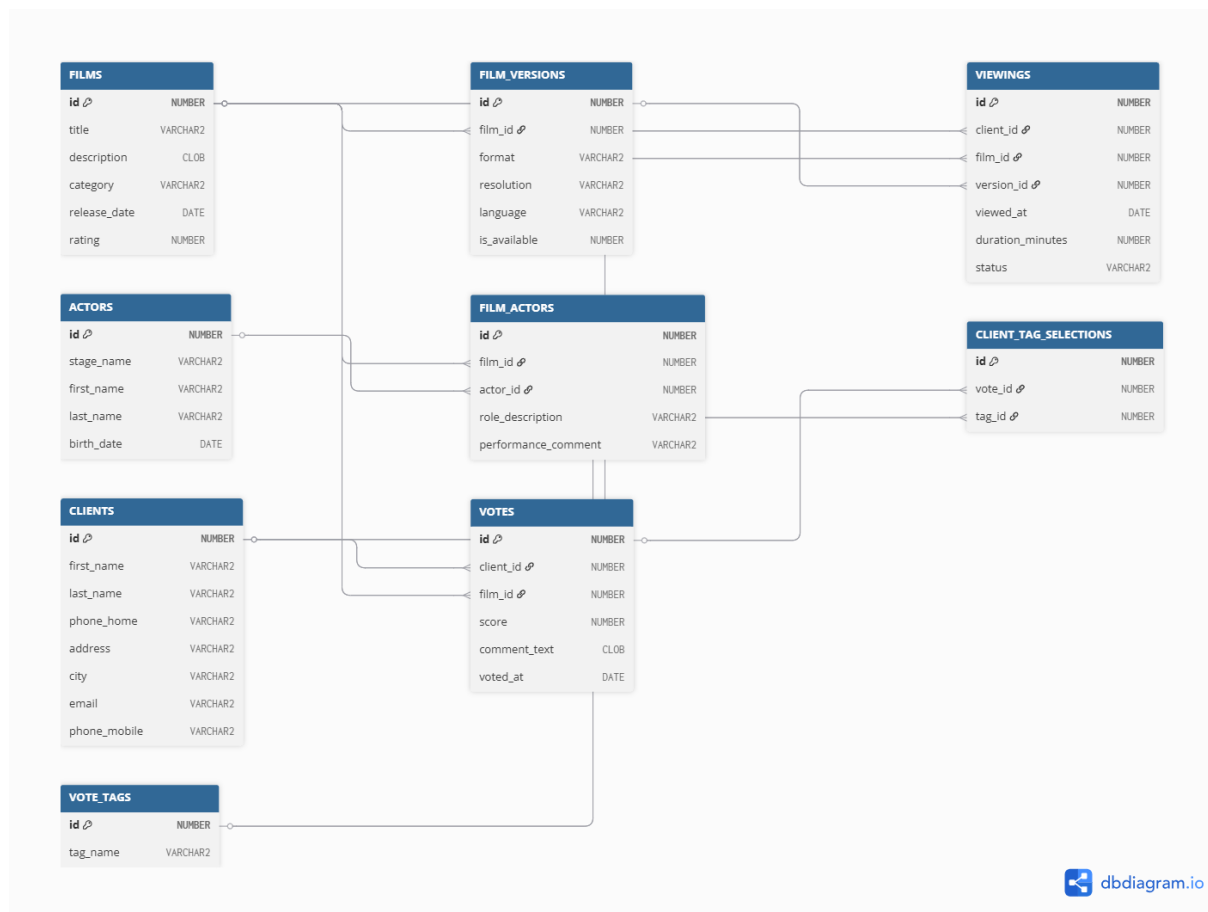


Figure 1: Diagrama Bazei de Date (Schema E-A)

Cum se vede din structura, FILMS e in centrul universului CineTrack. Un lucru asumat a fost folosirea integrala a cheilor primare sintetice. Datele din lumea reala (un nume de actor, un titlu de film) se pot schimba. Daca as fi pus pseudonimul actorului ca Primary Key, o corectura de nume din backend ar fi declansat cascade costisitoare de update-uri in tabelele legate. Astfel, pastrand ID-uri numerice invizibile pentru client, asigur o fundatie extrem de stabila.

Pentru integritate, am folosit toate armele oferite de Oracle. Pe langa PK, am pus ON DELETE CASCADE pe Foreign Key-urile din entitatile slabe (daca se sterge un film, dispar si versiunile lui video din FILM_VERSIONS sau comentariile din FILM_ACTORS). Foarte utila mi s-a parut constrangerea UNIQUE compusa. De exemplu, in FILM_VERSIONS am pus UNIQUE(film_id, format, language), pentru ca logic, un film nu are cum sa aiba incarcat de doua ori acelasi fisier video de aceeasi rezolutie si limba. Similar in VOTES, UNIQUE(client_id, film_id) e un blocaj absolut impotriva spam-ului cu voturi, garantand ca un user are o singura parere per film. Nu in ultimul rand, am abuzat (in sensul bun) de constrangerile CHECK (ex: rating intre 0 si 10, statusuri stricte pe vizionari), pentru ca

validarea din backend sau frontend nu e niciodata suficient de sigura fata de un INSERT direct din consola bazei de date.

5. Solutii de implementare curata in Oracle

Spre deosebire de alte platforme unde autoincrementarea se face prin Identity columns complicate sau triggere per tabela, am folosit o sintaxa curata, moderna: `DEFAULT sequence.NEXTVAL` direct la definirea coloanei. Pentru a pastra o flexibilitate maxima (in special utila la scripturile mele de drop & recreate din dezvoltare), am definit secventele inaintea tabelelor si le-am integrat elegant in CREATE TABLE:

```
CREATE SEQUENCE seq_films START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;

CREATE TABLE FILMS (
  id          NUMBER DEFAULT seq_films.NEXTVAL,
  title       VARCHAR2(255) NOT NULL,
  description CLOB,
  category    VARCHAR2(100) NOT NULL,
  release_date DATE,
  rating      NUMBER(3,1) DEFAULT 0,
  CONSTRAINT pk_films PRIMARY KEY (id),
  CONSTRAINT chk_films_rating CHECK (rating BETWEEN 0 AND 10)
);
```

Folosind aceasta metoda, in cazul in care la popularea din aplicatie nu trimit explicit un ID, Oracle stie sa si-l asigneze singur prin `seq_films.NEXTVAL`, dar ma lasa, de asemenea, sa fortiez manual anumite ID-uri (ex: `id=1, 'Inception'`) in scriptul de `populate.sql` pentru a putea lega corect cheile straine fara sa ghicesc valoarea de increment.

6. Triggere si logica de business nativa

Am vrut ca regulile grele ale afacerii sa fie protejate la cel mai jos nivel. Un exemplu clar e trigger-ul pentru validarea versiunii de film: daca un utilizator incearca sa adauge in istoric o vizionare, trigger-ul `trg_validate_viewing` verifica efectiv prin SELECT in tabela `FILM_VERSIONS` daca ID-ul acelei versiuni are setat `film_id`-ul corect. Daca nu, arunc `RAISE_APPLICATION_ERROR(-20002, '...')`.

Insa partea de care sunt cel mai multumit este modul in care am scris actualizarea automata a ratingului la filme in `trg_update_film_rating`. Initial am crezut ca va fi o lupta grea sa identific exact cand se face INSERT, UPDATE sau DELETE si sa leg actiunea corecta. Am gasit o solutie foarte eleganta intr-un singur trigger de tip AFTER ... FOR EACH ROW, folosind sintaxa conditionala DELETING:

```
CREATE OR REPLACE TRIGGER trg_update_film_rating
```

```

AFTER INSERT OR UPDATE OR DELETE ON VOTES FOR EACH ROW
DECLARE v_film_id NUMBER;
BEGIN
    v_film_id := CASE WHEN DELETING THEN :OLD.film_id ELSE :NEW.film_id
END;
UPDATE FILMS SET rating=(SELECT NVL(ROUND(AVG(score),1),0) FROM VOTES
WHERE film_id=v_film_id)
WHERE id=v_film_id;
END trg_update_film_rating;
/

```

Aceasta bucata de cod e super compacta si capteaza orice modificare. Extrage rapid `film_id`-ul corect, calculeaza media la zi prin subquery pe toata tabela de voturi a filmului si actualizeaza randul din `FILMS`.

7. Analiza Sentimentelor si Profilarea prin Proceduri Stocate

Daca aduceam toata informatia bruta pe retea spre Java, latentia ar fi omorat experienta utilizatorului. Asa ca am scris 5 proceduri stocate PL/SQL care agregheaza mii de log-uri si arunca afara un simplu `SYS_REFCURSOR`.

Spre exemplu, `proc_get_client_profile` intoarce nu doar istoricul filmelor, ci si o agregare puternica: cine e actorul preferat si care e genul preferat al utilizatorului. Pentru asta, am folosit interogari cu `FETCH FIRST 1 ROWS ONLY` aplicate direct pe subquery-uri ordonate cu functii `COUNT(*)`.

In plus, am conceput `proc_analyze_sentiment`, un algoritm integrat care functioneaza pe 3 niveluri in mod succesiv: preia media notelor, apoi numara cate tag-uri “pozitive” (ex: “emotionant”) vs “negative” a bifat clientul si, la final, foloseste `INSTR` pentru a scana la propriu clob-ul comentariului dupa cuvinte cheie (“excelent”, “dezamagitor”). Face un scor compozit si imi livreaza verdictul: `POZITIV` sau `NEGATIV`.

De asemenea, ca o faza pregatitoare pentru masuratori avansate, am scris `proc_cluster_clients`, unde utilizez functii analitice moderne din SQL, precum `RANK() OVER(PARTITION BY ... ORDER BY ...)`, pentru a grupa clientii (ex: ‘`ACTION_LOVER`’ sau ‘`DRAMA_FAN`’) pe baza categoriei dominante, ceea ce imi deschide drumul catre un motor colaborativ de recomandari foarte solid in viitor.

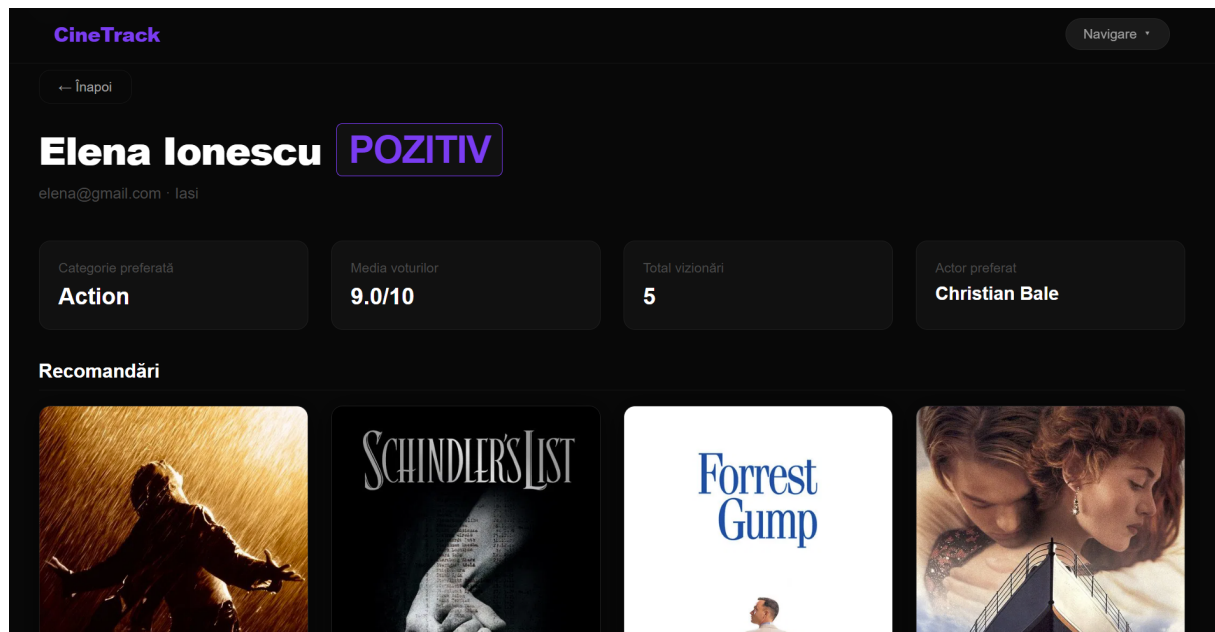


Figure 2: Profil Client generat nativ din Oracle: Sentiment calculation si Recomandari

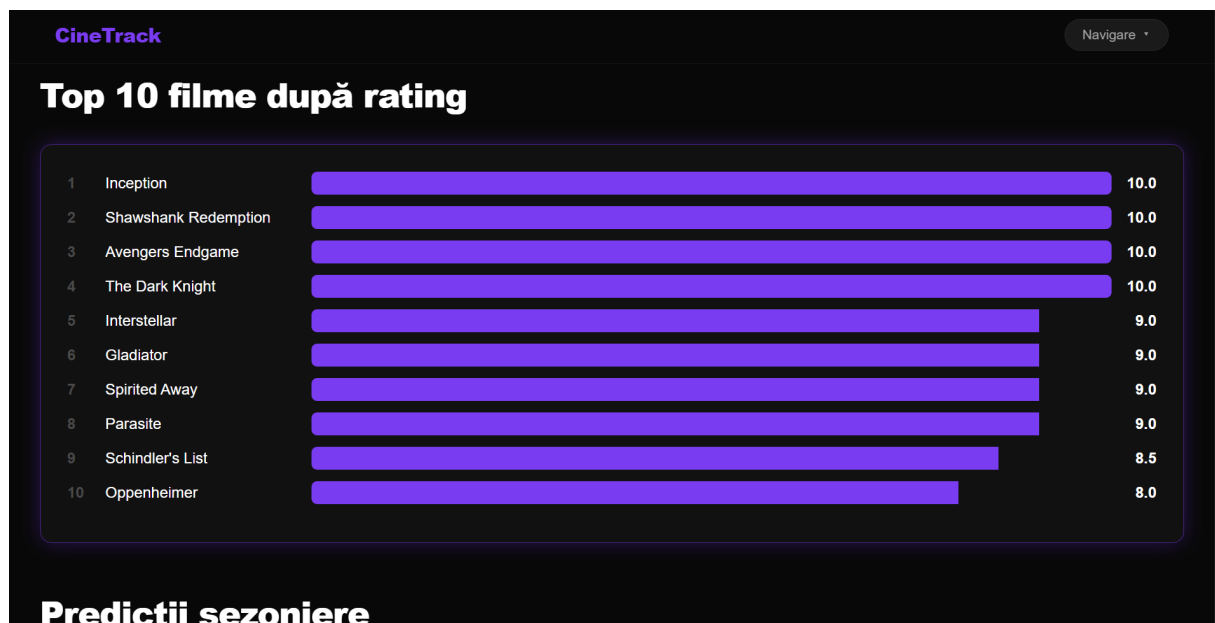


Figure 3: Top Filme - interogare statistica pe baza de date preluata in UI

8. Backend cu JDBC pur si prinderea exceptiilor PL/SQL

Aici m-am indepartat intentionat de framework-urile gen Hibernate/JPA. Am vrut sa fac conexiunile prin DAO-uri unde deschid personal `CallableStatement`-ul si generez obiectul intors prin `ResultSet`. Asta a venit cu mici ciudatenii rezolvate repede, gen folosirea obligatorie a constantei `oracle.jdbc.OracleTypes.CURSOR` (pentru `ojdbc11`), in loc de clasicul `Types.REF_CURSOR` din Java pur, care pica inexplicabil.

Cea mai mare victorie in arhitectura asta este securitatea datelor via exceptii. Cand cineva incearca sa dea un vot duplicat, trigger-ul meu oracle arunca eroare cu numarul -20001. In Java-ul meu scriu `if (e.getErrorCode() == 20001) throw new DuplicateVoteException`. Asta duce cererea in controller-ul meu global cu `@ExceptionHandler`, iar frontend-ul primeste in loc de un stacktrace urat, un simplu JSON de HTTP 409 Conflict pe care il randeaza elegant utilizatorului.

9. Implementare Frontend: Dark Mode curat si Reactivitate “Signals”

Fiind o platforma cu orientare media vizuala, trebuia ca prima pagina sa dea impresia unui sistem de streaming “wow”. Aici intervin posterele. Interesant e ca in `schema.sql` pe baza de date Oracle nu exista deloc o coloana `poster_url` — consider inutil sa salvez poze in tabele relationale brute, unde ar ocupa un spatiu masiv si ar degrada interogările. Posterele sunt integrate la nivel de aplicatie/DTO in functie de ID, sau preluate dintr-un bucket, legatura fiind asigurata pe baza de date curata de backend si randata ca atribut `[src]` dinamic in Angular.

Apoi am renuntat complet la observabilele obositoare din RxJS pe partea de cautare/-filtrare live, mergand all-in pe functionalitatea Signals din noul Angular 18. Definesc un semnal de cautare direct legat de bara de input, si un `computed()` derivat. Cum incepi sa tastezi, lista de pe ecran se filtreaza in cateva milisecunde doar client-side, posterele sarind corect in locurile lor, fara absolut nicio presiune extra asupra bazei de date Oracle.

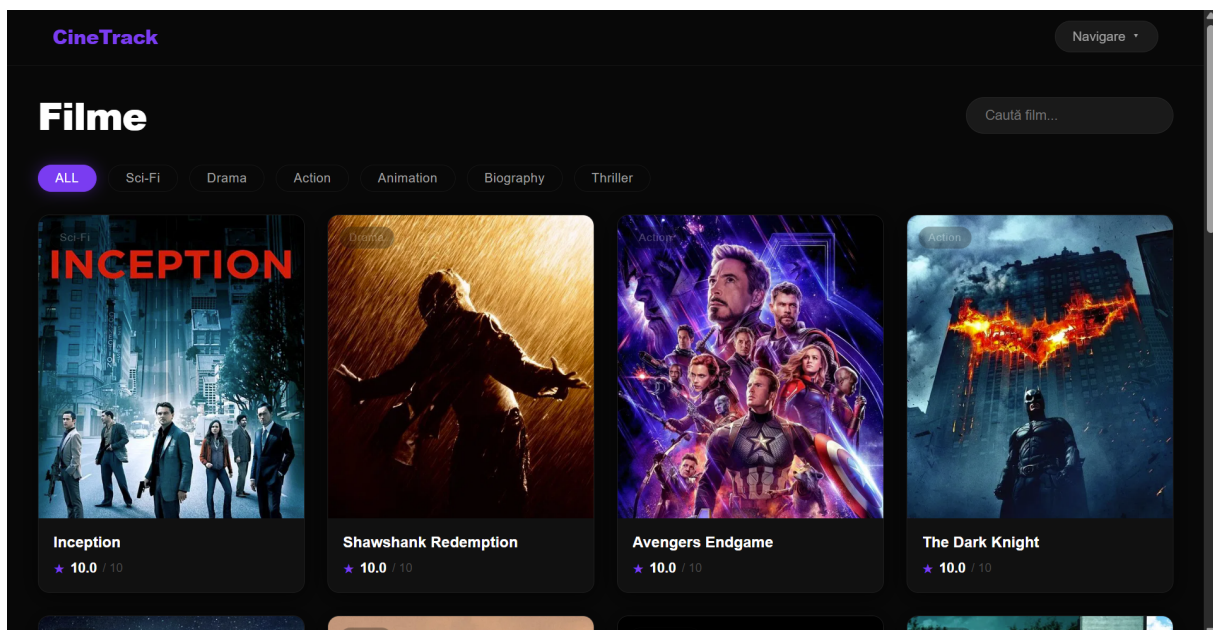


Figure 4: Pagina principala (Catalog) randata reactiv cu Signals si Dark Mode CSS pur

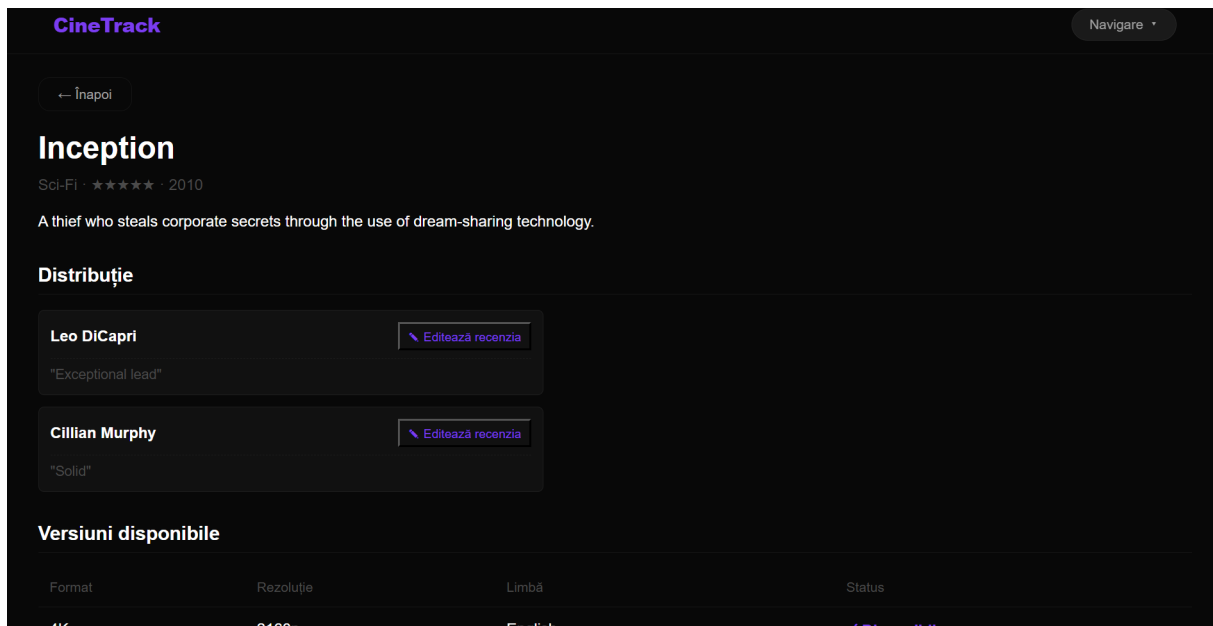


Figure 5: Interfata de vizionare: Detalii filme si distributie unde actorii primesc recenzii pe rol

Punctul final in decizia de UX a fost design-ul. Nu exista Bootstrap in acest proiect. Tot fisierul `styles.css` e munca manuala, cu un sistem solid de `:root` in care definesc variabile cromatice de baza (ex: `--surface: #111111`, `--accent: #7c3aed`). Cu un fond profund intunecat si butoane care raspund la `box-shadow` subtil, platforma respira aerul unei aplicatii cu adevarat native.

10. Popularea si validarea schemei

O baza de date Oracle, in ciuda triggerelor perfecte, e doar atat de buna ca si scriptul de populare. Am pregatit insert-uri ample pentru fiecare tabel, depasind norma de 15 linii. Secretul a stat in ordonarea minutioasa din scriptul `populate.sql`: am plecat cu radacinile (`FILMS`, `ACTORS`, `VOTE_TAGS`), adaugand `film_id` sau `actor_id` exact. Dupa care am inceput asocierile din `FILM_VERSIONS` si tranzactiile masive pe `VIEWINGS`, aducand la suprafata complexitatea analitica a aplicatiei mele.

11. Concluzii si reflectii personale

CineTrack a depasit stadiul de exercitiu si s-a transformat in certitudinea ca pot conecta cap-coada o idee abstracta intr-o platforma completa. Am realizat ca un backend robust e inutil daca nu e sustinut de triggere serioase care te previn sa incarci balast sau rating-uri false, la fel cum o pagina minunata in Angular va pica in nas daca datele care ii sosesc sunt dezorganizate sau procesate lenevitor din milioanele de randuri ale Oracle.

M-a surprins si incantat in acelasi timp felul in care SQL-ul pur poate deveni un mini limbaj analitic prin functiile de analiza text (`INSTR` pt cuvinte cheie) si functiile windowing (`RANK()` pt dominanta de clustere). Este fara indoiala cel mai matur sistem pe care l-am construit pe perioada cursurilor, si ma simt complet echipat sa justific si sa sustin fiecare linie din scriptul sau.

12. Provocari tehnice ascunse: Securitate, Performanta si Integrare

O mare parte din efortul acestui proiect nu s-a consumat doar pe logica din interiorul bazei de date, ci pe modul in care aceasta comunica cu exteriorul. Renuntarea la framework-uri de tip ORM a adus o mare libertate, dar si responsabilitati pe masura.

In primul rand, a trebuit sa rezolv problema vulnerabilitatilor. Fara un Hibernate care sa sanitizeze automat input-urile venite din frontend, a trebuit sa ma asigur ca absolut fiecare interogare scrisa in DAO-urile din Java (Data Access Objects) utilizeaza strict `PreparedStatement`. Construirea dinamica a query-urilor pentru bara de cautare a fost o lectie clara despre preventia atacurilor de tip SQL Injection. Toate datele introduse de utilizator au fost transmise bazei de date prin bind-parameters (folosind simbolul `?`), evitand cu strictete concatenarea bruta a string-urilor.

O alta problema reala de performanta a aparut la executarea repetata a procedurilor stocate. Am realizat rapid ca deschiderea si inchiderea unei conexiuni noi la Oracle pentru fiecare client care incarca pagina de catalog este o operatiune extrem de costisitoare, care blocheaza baza de date in conditii de trafic. Solutia nativa aplicata in Spring Boot a fost utilizarea unui Connection Pool (HikariCP). Acesta mentine in fundal un set de conexiuni active catre Oracle pe care le recicleaza si le imprumuta thread-urilor, scazand timpul de raspuns al aplicatiei de la cateva sute de milisecunde la un nivel aproape instantaneu.

La nivelul retelei, integrarea aplicatiei de Angular cu backend-ul m-a fortat sa inteleg si sa implementez corect politicile de CORS (Cross-Origin Resource Sharing). Deoarece frontend-ul rula local pe portul 4200, iar serverul de Java pe portul 8080, browser-ul bloca nativ orice cerere catre baza de date din motive de securitate. A trebuit sa configurez o clasa de `WebMvcConfigurer` in Spring Boot care sa permita accesul explicit pentru metodele GET si POST provenite din aplicatia de UI. De asemenea, maparea manuala a datelor din structura masiva a `ResultSet`-ului primit de la Oracle intr-un model strict tipizat in TypeScript a cerut o rigoare deosebita; o simpla litera gresita in numele unei coloane ducea la o interfata care afisa date sub forma de `'undefined'`.

13. Directii de dezvoltare pe viitor si scalabilitate

Desi platforma este in acest moment complet functionala si respecta cu strictete regulile de business impuse, exista metode clare prin care arhitectura ar putea fi imbunatatita pentru un mediu de productie real, cu zeci de mii de utilizatori.

Prima directie de optimizare vizeaza zona de Dashboard si statistici. In momentul de fata, graficele si topurile (precum "Top 10 filme dupa rating") sunt calculate on-the-fly prin interogari masive sau proceduri stocate la fiecare incarcare a paginii. La un volum urias de date, acest lucru ar deveni inefficient. Un upgrade logic la nivelul bazei de date Oracle ar fi transformarea acestor agregate in **MATERIALIZED VIEWS**. Aceste vizualizari materializate ar putea fi reincarcate asincron o data pe ora sau noaptea printr-un **DBMS_SCHEDULER**, servind astfel frontend-ului date precalculate cu efort zero pe procesor in timpul zilei.

A doua directie de dezvoltare naturala ar fi adaugarea unui strat real de securitate la nivelul utilizatorului. Desi baza de date asigura izolarea clientilor pe baza de **client_id**, platformei ii lipseste in prezent un mecanism de autentificare propriu-zis. Implementarea unui sistem bazat pe standardul **JWT (JSON Web Tokens)** ar reprezenta pasul final. In felul acesta, rutele de API ar fi blocate si protejate, iar identificadorul clientului nu ar mai veni dintr-un simplu parametru din frontend, ci ar fi extras criptografic direct din token-ul validat de server, asigurand o integritate totala a fluxului de date, de la browser pana in structurile din Oracle.